

ChatSocket

Aplicação de Chat em Tempo Real via Sockets TCP

Documentação da Aplicação e Manual de Usuário

Trabalho RC1 — Sockets

Disciplina: Redes de Computadores

Integrantes do grupo:

Ana Julia Lins Silva — SC3050661

Matheus Jardim de Ornellas — SC3051501

Munir Youssif Ferreira Abbas — SC3050777

Pedro Silva de Andrade — SC3050793

Link do repositório GitHub:

Link do vídeo-manual (YouTube):

1. Introdução

Este documento descreve a aplicação ChatSocket, desenvolvida como Trabalho RC1 da disciplina de Redes de Computadores. O trabalho propõe a implementação de uma aplicação cuja comunicação entre processos é realizada por meio de sockets — mecanismo de troca de dados em rede entre dois ou mais processos, na mesma máquina ou em máquinas diferentes interligadas por rede.

Optamos por desenvolver um chat multiusuário em tempo real, por ser uma aplicação que naturalmente exige a resolução de problemas centrais da programação com sockets: conexões simultâneas, concorrência entre múltiplos clientes, e a definição de um pequeno protocolo de aplicação para organizar a troca de mensagens.

1.1. Objetivo da aplicação

O ChatSocket permite que múltiplos usuários se conectem a um servidor central e troquem mensagens de texto em tempo real, semelhante a uma sala de bate-papo. Além do envio de mensagens livres, a aplicação oferece comandos especiais para listar usuários conectados e exibir ajuda.

1.2. Tecnologia utilizada

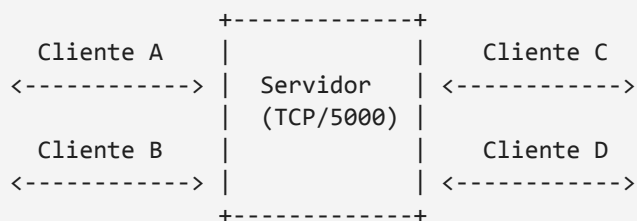
- Linguagem: Python 3.
- Mecanismo de comunicação: sockets TCP (módulo socket da biblioteca padrão, família AF_INET, tipo SOCK_STREAM).
- Concorrência: módulo threading da biblioteca padrão, para atender múltiplos clientes no servidor e para permitir envio/recebimento simultâneo no cliente.
- Sincronização: threading.Lock(), para proteger o acesso concorrente à estrutura de dados compartilhada que armazena os clientes conectados.
- Sem dependências externas: o projeto utiliza apenas bibliotecas padrão do Python (socket, threading, sys, datetime), não exigindo instalação de pacotes adicionais.

2. Arquitetura da aplicação

A aplicação segue o modelo cliente-servidor, um dos paradigmas mais utilizados em aplicações de rede. Nesse modelo, existe um processo servidor, que permanece em execução aguardando conexões em um endereço IP e porta conhecidos, e um ou mais processos cliente, que iniciam a conexão com o servidor.

2.1. Visão geral

Toda comunicação entre dois clientes é intermediada pelo servidor — não há conexão direta entre clientes. Quando um cliente envia uma mensagem, ela é recebida pelo servidor e redistribuída (broadcast) para todos os demais clientes conectados.



- Cada seta representa uma conexão TCP independente e persistente.
- O servidor recebe de um cliente e retransmite (broadcast) aos demais.
- Não há conexão direta entre os clientes.

2.2. Papel de cada arquivo de código

Arquivo	Responsabilidade
servidor.py	Cria e configura o socket do servidor; aceita conexões de múltiplos clientes (uma thread por cliente); conduz o handshake de apelido; processa mensagens e comandos; realiza o broadcast de mensagens entre os clientes conectados; mantém logs de eventos com data/hora; gerencia a entrada e saída de clientes de forma segura entre threads (com Lock).
cliente.py	Cria o socket do cliente e conecta ao servidor; participa do handshake inicial enviando o apelido escolhido pelo usuário; envia mensagens digitadas pelo usuário; recebe mensagens de outros usuários em uma thread separada, em paralelo à digitação; trata desconexões e erros de rede de forma controlada.

2.3. Concorrência: threads e Lock

No servidor, cada cliente conectado é atendido por uma thread independente (função `tratar_cliente`). Isso permite que o servidor receba mensagens de vários clientes ao mesmo tempo, sem que um cliente precise esperar o outro terminar de enviar sua mensagem.

Como múltiplas threads acessam simultaneamente o dicionário compartilhado que guarda os clientes conectados (`clientes`), o acesso a essa estrutura é protegido por um `threading.Lock()`. Isso evita condições de corrida (race conditions) — por exemplo, uma thread tentar enviar uma mensagem para um cliente que está sendo removido por outra thread no mesmo instante.

No cliente, a leitura do teclado (`input()`) é uma operação bloqueante: enquanto o programa espera o usuário digitar, ele não pode processar nada mais na mesma thread. Por isso, o recebimento de mensagens roda em uma thread separada (`receber_mensagens`), permitindo que mensagens de outros usuários apareçam na tela mesmo enquanto o usuário local está digitando.

3. Protocolo de comunicação

O TCP (socket SOCK_STREAM) garante a entrega confiável e ordenada de bytes entre as partes, mas não define, por si só, onde uma "mensagem" começa ou termina — ele transmite um fluxo contínuo de bytes. Por isso, construímos sobre o TCP um pequeno protocolo de aplicação, que define o formato e o significado das mensagens trocadas.

3.1. Handshake inicial (definição de apelido)

Assim que um cliente conecta, o servidor envia a string "NICK" como sinal solicitando um apelido. O cliente responde enviando o apelido escolhido pelo usuário, que é então validado (não pode ser vazio; se duplicado, recebe um sufixo numérico) e associado àquela conexão pelo restante da sessão.

Cliente	Servidor
--- connect() ----->	<----- "NICK" -----
--- apelido (ex: "Alice") ----->	<----- "[SERVIDOR] Bem-vindo..." -

3.2. Formato das mensagens

- Mensagens comuns (texto livre, não iniciado por "/"): redistribuídas a todos os outros clientes no formato [HH:MM] apelido: texto. O remetente recebe uma confirmação local no formato [HH:MM] Você: texto.
- Comandos (iniciados por "/"): interpretados pelo servidor e tratados separadamente (ver seção 3.3).
- Mensagens do sistema: prefixadas por "[SERVIDOR]", usadas para notificações de entrada/saída de usuários, respostas a comandos e avisos.
- Delimitador de mensagem: cada mensagem enviada pelo servidor termina com o caractere de nova linha (\n). Isso é importante porque, como o TCP não delimita mensagens automaticamente, sem um terminador consistente duas mensagens enviadas em sequência rápida poderiam chegar concatenadas em uma única chamada de `recv()` no cliente.

3.3. Comandos disponíveis

Comando	Função
/ajuda	Exibe a lista de comandos disponíveis.
/usuarios	Lista os apelidos de todos os usuários conectados no momento.
/sair	Encerra a conexão do cliente com o servidor de forma controlada.

3.4. Limitações conhecidas do protocolo

Por se tratar de um trabalho acadêmico com foco em sockets, algumas simplificações foram conscientemente mantidas e estão documentadas aqui:

- Tamanho de buffer fixo (1024 bytes): mensagens muito longas, ou várias mensagens pequenas enviadas em sequência muito rápida, podem ser entregues juntas em uma única chamada de `recv()`. O uso do terminador `\n` minimiza esse problema, mas uma implementação mais robusta poderia usar um protocolo de comprimento prefixado (enviar primeiro o tamanho da mensagem) para eliminar essa ambiguidade por completo.
- Sem criptografia: as mensagens trafegam em texto puro (sem TLS/SSL), adequado para fins didáticos, mas não recomendado para uso em produção.
- Sem persistência: o histórico de mensagens não é salvo em disco; ao reiniciar o servidor, todo o histórico da sessão é perdido.
- Apelidos não exigem autenticação: qualquer cliente pode se identificar com qualquer apelido disponível, sem senha.

4. Como executar a aplicação

4.1. Pré-requisitos

- Python 3.6 ou superior instalado.
- Nenhuma biblioteca externa é necessária (apenas módulos da biblioteca padrão do Python).

4.2. Iniciando o servidor

Em um terminal, no diretório onde está o arquivo `servidor.py`, execute:

```
python servidor.py
```

O servidor iniciará escutando em todas as interfaces de rede da máquina (0.0.0.0), na porta 5000, e exibirá um log de eventos no próprio terminal (novas conexões, entradas/saídas de usuários, mensagens trocadas).

4.3. Conectando um cliente

Em outro terminal, execute:

```
python cliente.py
```

Por padrão, o cliente tentará se conectar ao servidor em 127.0.0.1 (localhost), porta 5000. Para conectar a um servidor em outra máquina da rede, informe o endereço IP como argumento:

```
python cliente.py 192.168.1.10
```

Repita esse processo em terminais diferentes para simular múltiplos usuários conectados simultaneamente.

5. Manual do Servidor (servidor.py)

5.1. Visão geral

O servidor não exige interação direta do usuário além de iniciá-lo e, eventualmente, encerrá-lo. Ele é responsável por aceitar conexões, processar o handshake de apelido, redistribuir mensagens entre os clientes e manter um log detalhado de tudo o que acontece.

5.2. Iniciando o servidor

Execute `python servidor.py` no diretório do projeto. Se a porta 5000 já estiver em uso por outro processo, o servidor exibirá uma mensagem de erro e não iniciará — nesse caso, finalize o processo que está usando a porta ou edite a constante `PORT` no início do arquivo.

5.3. Lendo os logs do servidor

Cada linha de log é prefixada com o horário do evento. Os principais eventos registrados são:

- Nova conexão: `<endereço>` — um novo cliente TCP se conectou (ainda antes do handshake de apelido).
- `'<apelido>'` entrou no chat. — o handshake foi concluído com sucesso.
- Clientes conectados: `<N>` — total de clientes ativos no momento.
- `[HH:MM] <apelido>: <mensagem>` — mensagem de chat recebida e redistribuída.
- `'<apelido>'` saiu do chat. — cliente desconectou (seja por `/sair`, fechamento da janela ou queda de rede).
- Conexão recusada de `<endereço>`: limite de `N` clientes atingido. — o servidor atingiu o limite máximo de clientes simultâneos (constante `MAX_CLIENTES`) e recusou uma nova conexão.

5.4. Encerrando o servidor

Pressione `Ctrl+C` no terminal do servidor. Antes de encerrar, o servidor envia uma mensagem de aviso a todos os clientes conectados ("O servidor está sendo encerrado") e fecha todas as conexões de forma organizada.

5.5. Parâmetros configuráveis

Constante	Valor padrão	Descrição
HOST	0.0.0.0	Endereço em que o servidor escuta. 0.0.0.0 significa todas as interfaces de rede disponíveis.
PORT	5000	Porta TCP utilizada pelo servidor.
MAX_CLIENTES	10	Número máximo de clientes simultâneos aceitos pelo servidor.
BUFFER	1024	Tamanho, em bytes, do buffer de leitura de cada <code>recv()</code> .

6. Manual do Cliente (cliente.py)

Esta seção apresenta, de forma revisada e ampliada, o manual de uso do cliente do ChatSocket.

6.1. Requisitos

- Python 3.6 ou superior instalado no computador.
- Conexão de rede com o servidor (local ou remota).
- O arquivo cliente.py.
- O servidor deve estar em execução antes de iniciar o cliente.

6.2. Executando o cliente

6.2.1. Forma padrão (servidor local)

Abra um terminal no diretório onde está cliente.py e execute:

```
python cliente.py
```

Por padrão, o cliente tentará conectar ao servidor no endereço 127.0.0.1 (localhost), porta 5000.

6.2.2. Conectando a um servidor remoto

Para especificar um endereço IP diferente, passe-o como argumento:

```
python cliente.py 192.168.1.10
```

A porta permanece 5000 (não configurável por linha de comando nesta versão; pode ser alterada editando a constante PORT no arquivo).

6.2.3. Exemplo de saída inicial

```
=====
ChatSocket - Cliente TCP
=====
Servidor: 127.0.0.1:5000
=====
Conectando ao servidor...
Digite seu apelido:
```

6.3. Primeiros passos

6.3.1. Escolha do apelido

- Após conectar, o servidor solicitará um apelido.
- Digite um nome de até 20 caracteres.
- Apelidos vazios não são aceitos.

- Se o apelido já estiver em uso por outro cliente conectado, o servidor atribuirá automaticamente um sufixo numérico para diferenciá-lo.

6.3.2. Entrando no chat

Após confirmar o apelido, você verá:

```
Conectado! Olá, [seu_apelido]! 🎉
Digite suas mensagens e pressione Enter para enviar.
Use /ajuda para ver os comandos disponíveis.
-----
Você:
```

A partir daqui, você já pode digitar mensagens.

6.4. Enviando e recebendo mensagens

- Mensagens comuns: qualquer texto que não comece com "/" será enviado a todos os participantes.
- Formato exibido: [HH:MM] apelido: texto.
- Sua própria mensagem aparece com a marca "Você:".

Exemplo:

```
Você: Olá pessoal!
[14:30] João: Olá pessoal!
[14:31] Maria: Oi, João!
```

O cliente exibe a linha "Você:" automaticamente após cada envio, facilitando a digitação contínua.

6.5. Comandos especiais

Digite qualquer um dos comandos abaixo no início da linha, sempre precedidos por barra (/).

Comando	Descrição
/ajuda	Exibe a lista de comandos disponíveis.
/usuarios	Mostra os apelidos de todos na sala.
/sair	Encerra sua conexão e sai do chat.

Ao enviar /sair, o cliente exibe uma mensagem de despedida e fecha a conexão.

6.6. Comportamento em caso de problemas

6.6.1. Servidor não encontrado

Se o servidor não estiver em execução no endereço/porta configurados, o cliente mostrará:

```
[ERRO] Servidor não encontrado em 127.0.0.1:5000
```


→ Verifique se o servidor está rodando.

Solução: inicie o servidor (servidor.py) antes do cliente, ou corrija o endereço IP informado.

6.6.2. Conexão perdida durante o chat

- Se o servidor cair, o cliente exibirá: [INFO] Servidor desconectado inesperadamente.
- Se a rede do próprio usuário falhar, mensagens de erro aparecerão e o cliente tentará encerrar de forma controlada.

6.6.3. Apelido muito longo

Caso o apelido digitado tenha mais de 20 caracteres, o cliente avisará e pedirá novamente:

[AVISO] Apelido muito longo. Use até 20 caracteres.

6.6.4. Encerramento forçado (Ctrl+C)

Pressione Ctrl+C a qualquer momento. O cliente exibirá [INFO] Encerrando... e fechará a conexão corretamente.

6.7. Dicas e observações

- O cliente usa uma thread separada para receber mensagens enquanto o usuário digita — as mensagens de outros participantes aparecem automaticamente, sem atrapalhar a digitação em andamento.
- Mensagens vazias (apenas Enter) são ignoradas.
- Todo texto enviado é interpretado como UTF-8.
- Se o terminal for fechado abruptamente, o servidor detecta a queda de conexão e remove o usuário da sala automaticamente.

6.8. Personalização avançada (para usuários com conhecimento técnico)

Embora o cliente não aceite argumentos de linha de comando para a porta, é possível alterar a constante PORT diretamente no arquivo cliente.py (deve coincidir com a porta configurada no servidor). Também é possível alterar o tamanho do buffer (BUFFER), embora isso não seja recomendado sem entender as implicações descritas na seção 3.4 deste documento.

7. Considerações finais

O ChatSocket cumpre o objetivo proposto pelo Trabalho RC1: implementar uma aplicação cuja comunicação entre processos depende de sockets, demonstrando na prática conceitos centrais de redes de computadores, como o modelo cliente-servidor, sockets TCP, concorrência com múltiplas conexões simultâneas e a construção de um protocolo de aplicação sobre o transporte oferecido pelo TCP.